Networking

Networking Goals:

- Reliable transport; hostname communication; many ports/connections; secure layers
- Hardware: single Network Interface Controller (NIC), unreliable, MAC-only, no security
- OS layers bridge hardware and application needs

Network Stack

- **5 Layers:** App / Transport / Network / Link / Physical
- **Encapsulation:** frames → packets → segments
- **Ethernet:** connects devices within Local Area Network (LAN) using physical cables
- **Address Resolution** Protocol **(ARP):** find the physical (MAC) address for a given IP address on a local network (LAN)
- **Internet Protocol (IP):** how data is sent and received between devices on a network; routing; multi-hop (multiple servers)
- **User Datagram Protocol (UDP):** fast, lossy, dangerous, unordered (e.g., gaming, streaming)
- **Transmission Control** Protocol **(TCP):** reliable, ordered, seq#, ACKs, Head of Line (HOL) blocking, congestion-controlled (e.g., file transfers, email, browsing)

Network Layer (IP): multi-hop routing, forwarding (facilitate movement of packets to destination), IP address construction

Transport Layer: end-to-end, ports, multiplexing (multiple sources into one stream)

Sockets: represented by a file descriptor in OS and **.NET** framework provides classes to manage network sockets

- Server: socket → bind → listen → accept
- Client: socket → connect
- Read/write semantics, more failure modes

Security

Security Goals:

- Security = desired properties vs. adversary
- Policy: rules; Mechanisms: enforcement (hw, sw)
- Least privilege; analyze goals + threat model

Bad Policies: overbroad perms enable privilege chain escalations (teacher → superintendent example)

Bad Threat Models:

- Kerckhoff's Principle: system secure even if fully known
- Obscurity != Security

Bad Mechanisms:

- Bugs = security holes
- Buffer overflow: out-of-bounds write → overwrite return addr → control hijack

Passwords:

- Never store in plaintext
- Hash: maps a bit string of any size to a bitstring of length d; collision resistance; one-way
- Salt: random per-password, prevents rainbow tables, store (salt + hash)

File Systems

FS Abstractions / Layers

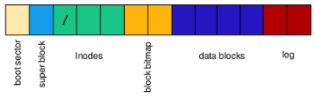
- Layers: FD → Path → Directory → INode → Logging → Buffer Cache → Disk
- Abstractions: naming, protection, isolation (locks), multiplexing, sharing, byte-level interface

FS API Design

- Challenges: crash recovery, performance, sharing API, security, abstraction (pipes/devices/proc), distributed FS (Andrew FS (access shared files from different computes), Network FS (access files over a network similar to a local drive), HDFS (Hadoop Distributed FS (large files across clusters of computers)
- API Choices
  - Granularity: files / virtual disks / DBs
  - Content: byte-array / records / binary trees
  - Naming: hierarchical (tree-like structure) vs. flat (unique identifier)
  - Sync: none / locks / txns / MVCC (multi-version concurrency control)
- Disk Tradeoff: block FS vs. sector disk, cost throughput varies by medium

Disk Layout: FS unit = block; dick unit = sector

INodes: metadata for on-disk content



- Tracks type, link count, size
- 12 direct block ptrs + 1 indirect block ptr
- Indirect block = ptrs to more data blocks
- Multiple links → inode needed to track shared object
- Directories store dirents (directory entry): (filename + inode #)
- Inodes cached in memory to avoid disk hits

Directories

- Directory is a file with dirents
- User cannot write directly; only kernel modifies
- Dirent = (name + inode #)

Buffer Cache

- RAM cache of disk blocks (fast reads)
- Constraints: only one copy per block; single-thread access; must lock to prevent races
- Cache hot blocks; eviction for limited RAM

File Descriptors

- FD = index into per-process FD table → file struct
- Syscalls take FD → kernel resolves file struct
- Conventional FDs: 0 stdin, 1 stdout, 2 stderr

Ordering / Atomicity

Problem:

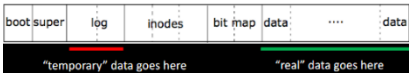
- Crash during write → inconsistent FS
- Inconsistency: free block in us, dir entry w/o data, dangling pointers
- Disk writes must be ordered + atomic
- Flush = force all pending ops to disk; enforces ordering

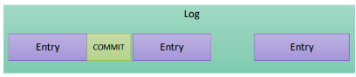
Shadowing: maintain two copies: curr + shadow; shadow bit selects version

- Process: copy current → shadow; write update; flush; atomically flip bit
- Crash Cases: crash after flip; new version
- Problems: 2x space, 2+ writs per update, shadow block separate → overhead huge

Full Logging: log describes new data; log takes precedence over data

- Process: write log entry → write 'done/commit' → apply to data blocks
- Crash Cases:
  - Crash before commit: discard log entry
  - Crash after commit: replay log





- Properties: atomic commits (commit block per entry), ordered via flush
- Pros: sequential log writes, good performance, atomicity, recovery from log
- Cons: double writes, log must be merged, slow metadata ops, more flushing

Logging w/ Checksum: checksum validates committed entries; sequential entries must be committed in order

- Benefits: no flush after every write; allow out of order physical writes while preserving logical ordering
- Commit: write entries; write checksum; committed iff checksum matches and previous entries committed
- Recovery: on crash, replay up to first uncommitted entry

Log Merge:

- When log is full: for each committed entry, overwrite data, then clear log
- Crash during merge: replay fro m state to first uncommitted entry, then clear

Ordered Mode: log only metadata; write data directly

- Process: write data block (free block) → flush; write metadata to log → flush; write commit + update inode/blocklist
- Double Flush? Ensure metadata never points to garbage; valid pointers; avoid committing nonexistent data
- Crash Cases:
  - Before commit: inode/blocklist unchanged → free block safe
  - After commit: inode correct
- Pros: lower overhead than full logging
- Cons: Still requires ordering → slower writes than unconstrained mode

POSIX FS Interface: IEEE std defining consistent FS behavior across Oses

- Guarantees: open/close/read/write semantics, FDs, blocking behavior, err codes, perms, links, directory ops, fsync, atomic rename
- Ensures interoperability + predictable behavior for multi-platform apps
- Core Goals: portability, compatibility, std syscalls, consistent file metadata semantics (mtype/ctime/atime), well-defined atomicity rules

Distributed Systems

Overview

- Many networked nodes acting as one system; abstraction, protection, rss management, isolation
- Goal: multi-PC OS illusion (single-system semantics)

Challenges: consensus (nodes w/ diff state, clocks); fault tolerance (unreliable channels, partitions)

MapReduce: Data too large for one machine → split across cluster

- Map: transform, filter, group
- Reduce: aggregate mapped results into final output
- Word count example: map(word → 1), reduce(sum)
- **Pros:**
  - Scales horizontally; auto-parallelization

- Fault tolerant (tasks can be re-run)
- Simple abstraction over huge datasets
- Data-local computation reduces network I/O

- **Cons:**
  - Requires synchronous, high-bandwidth cluster
  - Poor for cross-datacenter or high-latency networks
  - Not suitable for low-latency or transactional workloads
  - Limited communication model (map → shuffle → reduce only)
  - Inefficient interative algorithms (restarts each stage)

Two Generals Problem
- Unreliable channel: messages can be lost
- Need mutual ACK to attack
- Infinite ACK regression → impossible
- Takeaway: mutual certainty impossible over unreliable channels → consensus is hard

FLP Theorem
- Pure asynch system + possible node failure → deterministic consensus impossible
- No bounds on time → cannot distinguish show vs dead
- Motivates adding synchrony assumptions

Consensus via Timeout: add timeouts to break FLP assum. → partial sync
- Assume messages normally arrive within some time bound
- If time out expires, treat peer as failed → make progress
- System eventually becomes synchronous enough to terminate
- Downsides: slow network = crash; spurious leader elections; instability; difficult at scale

Raft

Goals:
- Reach consensus on a replicated log; maintain identical logs on majority
- Provide leader-based, easy-to-understand consensus

Leader Election
- Nodes start as followers
- If follower doesn't hear from leader → timeout → becomes candidate
- Candidate requests votes; majority → leader
- Term increments each election; stale leaders step down

Log Replication
- Leader appends client entry to its log
- Sends AppendEntries RPC to followers
- Entry committed once stored on a majority
- Followers apply entries in order to state machine

Safety
- At most one leader per term
- Leader's log is the source of truth; followers match it
- Log-matching property: if same index + term, all preceding entries identical
- Leader only commits safe on majority → ensures no divergent histories

CAP Theorem

Partition Tolerance required (real networks partition). Must choose **CA** or **AP**.
- Consistency (C): all nodes see the same data
- Availability (A): system responds despite failures
- Partition Tolerance (P): continues operating despite partitions

CP: keep consistency; lose availability during partition (zookeeper, raft)

AP: keep availability; allow divergence during partition (Dynamo, NoSQL)

Readings

The UNIX Operating System (Video)
- High-level overview of UNIX principles: simple interfaces, modularity, and composability ("small tools doing one thing well").
- Emphasizes the **kernel's role**: process management, file system, device interfaces.
- Shows how early UNIX established ideas still present today (pipes, hierarchical FS, process model).

The Night Watch
- Explains how the OS acts as a **supervisor** that protects processes from each other.
- Highlights kernel responsibilities: memory isolation, scheduling, hardware control.
- Stresses that **user processes should not trust each other**, and kernel must assume they are buggy or malicious.
- Reinforces the principle: *user code cannot directly touch hardware; all* access *goes through the kernel*.

K&R Readings (2.9, 5.1, 5.5, 6.4)

(Useful for x86, pointers, and low-level OS code understanding.)
- **2.9:** Bitwise operations — essential for manipulating flags, masks, hardware registers, and page table bits.
- **5.1:** Pointer basics — how C pointers represent memory addresses directly (important for OS memory layouts).
- **5.5:** Pointer arithmetic — stepping through arrays, stacks, and buffers; matches how kernel iterates page structures.
- **6.4:** Switch statements and jump tables — similar to syscall dispatch tables and interrupt vector tables.

How to Read a Paper (Keshav)
- Three-pass method:
  1. **First pass:** get the big picture.
  2. **Second pass:** understand methods at a moderate level.
  3. **Third pass:** deep understanding + ability to recreate work.
- Key skill for OS papers: extracting **problem, mechanism, evaluation**, and identifying assumptions.
- Helps avoid getting lost in implementation details.

Scheduler Activations

(*Effective Kernel Support for User-Level Management of Parallelism*)
- Bridges the gap between user-level threads and kernel scheduling.
- Kernel informs runtime of events (blocking, preemption) via **activations**, so user-level scheduler can manage threads correctly.
- Goal: get **flexibility of user threads** with **correctness + preemption awareness** of kernel-managed threads.
- Avoids problems where blocking syscalls freeze all user threads.

The Design Philosophy of the DARPA Internet Protocols
- Core design: **survivability** — network must continue working despite failures.
- Led to end**-to-end principle**: reliability, ordering, correctness implemented at the endpoints, not inside the network.
- Internet favored **flexibility and heterogeneity** over strict performance guarantees.
- Explains why IP is "best-effort" and simple, pushing complexity to higher layers (TCP, applications).
- Helps contextualize why TCP handles congestion, retransmission, flow control, etc., not routers.

Confused Deputy Problem
- A "deputy" program has its own privileges but also acts on behalf of users.
- If it accidentally uses *its own* authority instead of the user's, it can be tricked into misusing its privileges.
- Root cause: **ambient authority** (permissions coming from global namespaces like filenames).
- Solution: **capabilities** — explicit, unforgeable references that combine identity + permission.
- OS takeaway: APIs should avoid relying on global names to imply authority; pass explicit capabilities instead.

KeyKOS (Capability-Based OS)
- Pure capability-based OS: no ambient authority.
- All access occurs through capabilities that are securely unforgeable.
- Strong isolation using "domains" + message-passing.
- Persistent objects allow the system to recover state after crashes.
- Demonstrates least-privilege enforcement through the OS architecture itself.

Reflections on Trusting Trust
- A compiler can be modified to secretly inject backdoors into programs it compiles.
- Even if the source code looks clean, the compiler can insert malicious code into future compiler binaries too.
- Therefore: **you cannot trust software based solely on source** inspection.
- Reinforces need for verifiable toolchains (reproducible builds, signing, hardware roots of trust).
- Supports idea of a small, inspectable Trusted Computing Base (TCB).

Read-Copy Update (RCU)
- High-performance synchronization for read-heavy workloads.
- Readers**:** run without locks; see consistent snapshots.
- **Writers:** create updated copies, then atomically publish them (pointer swap).
- Old versions freed only after all pre-existing readers pass a **grace period**.
- **Pros:** extremely scalable, no blocking for readers.
- **Cons:** memory reclamation is delayed; writer logic is more complex.

Time, Clocks, and the Ordering of Events in a Distributed System (Lamport)

- Defines **happens-before (→)**: A causally influences B → A must come before B.
- Introduces Lamport **logical clocks**: each event gets a monotonically increasing timestamp.
- If A → B, then LC(A) < LC(B).
- Provides a consistent ordering without synchronized physical clocks.
- Limitation: cannot detect concurrency (two independent events may appear ordered arbitrarily).

Raft (Additional Reading Notes)
- **State Machine Safety:** once an entry is committed, it can never be overwritten — even after leader changes.
- **Commit Index:** leader marks entries committed once replicated on a majority; followers update via AppendEntries.
- **Leader Completeness:** any committed entry must appear in the logs of all future leaders.
- **Client Semantics:** clients send requests to the leader only; retry on timeout; use unique request IDs to prevent duplicate execution.

Additional Information

Networking — Additional Essentials
TCP 3-Way Handshake: SYN → SYN-ACK → ACK. Establishes seq numbers and reliability.
TCP Teardown: FIN/ACK on each direction; TIME_WAIT prevents old duplicate packets corrupting new connections.
Socket States: LISTEN, SYN_SENT, SYN_RCVD, ESTABLISHED, FIN_WAIT1/2, CLOSE_WAIT, TIME_WAIT.
Nagle's Algorithm: merges small writes → more throughput, more latency.
Delayed ACKs: waits briefly before sending ACK → may cause stalls with Nagle.

Security
UID, GID, EUID: determine identity and effective privileges of processes.
File Permission Bits: rwx for user/group/other.
setuid Programs: run with file owner's privileges → powerful but dangerous if buggy.
Access Control Lists (ACLs): per-file fine-grained permissions beyond rwx.
Kernel Attack Surface: syscalls, device drivers, parsing user inputs → common vulnerability sources.

File Systems
Hard Links vs Soft Links:
• Hard link → another dirent pointing to same inode (same FS only).
• Soft link → path to another file; can break; can cross FS boundaries.
Free Space Management:
• Bitmaps (efficient scanning)
• Free lists (simple but slower).
FS Consistency Checking (fsck): repairs mismatched link counts, orphaned inodes, incorrect free-block maps.
Directory Locality: FS try to place same-directory files near each other to reduce seek time.

Ordering & Atomicity
Write Barriers: enforce ordering constraints for journaling so metadata never points to unflushed data.
Atomicity Limits: only small writes (< block size) *might* be atomic; multi-block writes may be torn after crash.
mmap + msync: memory-mapped writes must use msync to ensure persistence; ordering not guaranteed without it.

Distributed Systems
Primary–Backup Replication: leader handles updates; backups apply them in order (fits Raft's log replication).
Failure Types:
• Crash failures → node stops responding (Raft model).
• Byzantine failures → arbitrary/malicious behavior (not handled by Raft).
Heartbeats: leader sends periodic empty AppendEntries to maintain authority and detect failures.
Client Semantics: clients retry timed-out requests and include unique IDs to avoid duplicate execution.

General OS Concepts
Cooperative vs Preemptive Scheduling:
• Cooperative → threads voluntarily yield; simple but unsafe.
• Preemptive → timer interrupts force context switches; standard in modern OS.
Blocking vs Non-blocking I/O:
• Blocking waits; non-blocking returns immediately; async I/O gives event-based notifications.
Cache Coherency: OS must use atomic instructions + memory barriers on multicore systems.
Kernel Preemption: disabled inside critical sections or while holding spinlocks; allowed elsewhere for responsiveness.

Practice questions

Virtual Memory

What operating system abstraction are we enabling or supporting with the ability to oversubscribe physical memory?
Page swapping to disk. Virtual memory allows each process to see a large, private address space even if physical RAM is limited. Oversubscription depends on swapping inactive pages to disk.

Why is exact LRU suboptimal in an OS like xv6? What information is needed, and what is the biggest overhead?
Exact LRU requires tracking every memory access. The OS would need to record timestamps or maintain an ordered list of page accesses.
The biggest overhead is trapping into the kernel **on every memory access**, even for pages already in memory. This creates massive context-switch overhead.

Describe the Accessed bit states and all state transitions used by the Clock Algorithm.
States:
0 = not recently accessed
1 = recently accessed
Transitions:
- On access: 0 → 1
- On access: 1 → 1
- When inspected by clock hand during eviction: 1 → 0

Which state is most desirable to the Clock Algorithm when choosing a page to evict?
The Accessed bit = 0, meaning "not recently used." This is the eviction candidate.

Networking

Explain the downsides of using TCP for your friend's messaging app. Give at least two.
1. Head-of-line blocking — one lost packet delays the entire stream.
2. Connection setup overhead (3-way handshake) — inefficient for mobile and real-time media.
   Additional possible disadvantages: conservative congestion control, NAT traversal issues, large per-connection state.

Explain how your friend could use UDP for his app, and what the advantages are.
He can implement reliability in the application layer: sequence numbers, retransmissions, jitter buffers, FEC, custom congestion control.
Advantages: low latency, no head-of-line blocking, no connection setup, full control over reliability, good NAT traversal. Ideal for real-time audio/video.

Explain which parts of the app should use TCP vs. UDP, and why.
UDP: voice and video (low latency, tolerate loss).
TCP: text chat, metadata, file transfers (need reliable, ordered delivery).

Security
If hackers steal hashed passwords from one system and salts from another, can they use a rainbow table to log in? Why or why not?
No. Rainbow tables cannot be reused across different salts. Each salt requires its own table. The attackers would need to brute-force individually.

Identify the vulnerability in cypher() and give the solution.
Vulnerability:
strcpy(buffer, input) allows buffer overflow since no bounds checking is performed. This can overwrite the return address and lead to code execution.
Fix:
Use strncpy() or manually enforce bounds.

File Systems
Why did video uploads stop even though only 1.7GB of 3GB is used?
The filesystem ran out of **inodes**, not disk space. Without inodes, no new files can be created.

Why did writes slow down after adding the 40-video combine feature?
Full synchronous journaling forces two disk writes per modification. A 2GB file being updated triggers huge numbers of forced syncs, causing slowdown.

Describe 2 ways to modify the filesystem to increase write speed.

1. Use **ordered journaling** (metadata-only) to reduce write amplification.
2. Use hardware tiering: small files → SSD, large sequential files → HDD.

Are all flush operations necessary in the intern's proposed sequence? What FS feature lets us reduce them?
No. Full journaling ensures atomicity.
Journaling (write-ahead logging) means fewer flushes are required because the journal commit determines atomicity.

How much space can the inode pointers index? (Given 4KB blocks, 4-byte pointers)
Direct (12 pointers):
12 × 4KB = **48 KB**
Indirect:
1024 entries × 4KB = **4 MB**
Double-indirect:
1024 × 1024 × 4KB = **4 GB**
Triple-indirect:
1024 × 4 GB = **4 TB**

Is the intern's filesystem design optimal? Why or why not?
No. Triple-indirect pointers allow up to 4TB per file, which is far beyond the app's maximum 2GB file size. This wastes inode space and adds overhead. A simpler design or extent-based system is better.

Threading Models
Imagine you are asked to create a Deep Learning (DL) Training application that runs on a libos system with only 32 cores. This application is very computationally intensive and runs many compute bound threads (> 32). Occasionally some threads make calls to the following IO bound services:
State two advantages and two disadvantages of using a 1:1 threading model. Why is it undesirable here?
Advantages:

1. True parallelism on all cores.
2. Kernel handles blocking I/O and scheduling.

Disadvantages:

1. Too many threads → heavy context switching.
2. Higher memory and kernel overhead per thread.

Undesirable: The app has many more compute threads than cores, so 1:1 causes thread thrashing.

Would an N:1 threading model make sense? Give two advantages if yes or two disadvantages if no.
No.
Disadvantages:

1. No parallelism — only one kernel thread, terrible for compute-bound DL.
2. If any thread blocks in the kernel, all threads block.
3.

State two advantages of using a hybrid M:N threading model.

1. True parallelism with reduced kernel overhead.
2. Custom user-level scheduling enhances compute performance.

State the main disadvantage of using a hybrid M:N threading model.
If a kernel thread blocks, all user threads mapped to it are stalled.
What is the most performant assignment of user threads to kernel threads? Explain how it fixes the main disadvantage.
Use **32 kernel threads** (one per core).
Map compute-bound user threads evenly across these 32 kernel threads.
Use a separate, small set of kernel threads for I/O-bound threads.
This isolates blocking operations to specific kernel threads, preventing them from stalling compute threads.

Race Conditions
How many outcomes can occur in Too Much Milk? List and explain.

```
Check Fridge:
    if no milk:
        go to store
        buy milk
        put milk into the fridge
```

Possible outcomes: **1 or 2.**
1 (only one buys milk):
One checks the fridge after the other returns, or one arrives at the fridge later and sees milk.
2 (both buy milk):
Both see the fridge empty before either returns.

In lecture, we discussed how to address tissue by using an atomic exchange operation.

```
// atomic swap (xchg) {
        t = *y;
        y* = x;
        return t;
}
```

We used this xchg() primitive to solve the "Too Much Milk" problem as follows:

```
void get_ fridge_access) {
        r = 1;
        while (r != 0) {
                r = xchg(&lock, 1);
        }
}
```

List all possible (lock, r) states.
(0, 1) – attempt when lock free
(1, 1) – attempt when lock held
(1, 0) – success, lock acquired

Which state is a success state?
(1, 0)

Describe two possible transitions that lead to success.
(0,1) → (1,0): lock was free; thread acquires lock.
(1,1) → (0,1) → (1,0): thread spins until lock released; then acquires lock.

What primitives could make acq() more resource-efficient?

1. Sleep/block primitive (e.g., futex wait)
2. Wakeup/signal primitive (e.g., futex wake)

These avoid spin-waiting.

Is it appropriate to use these primitives in the interrupt handler? Why or why not?
No. Interrupt handlers must not block or sleep; they must complete quickly. Only spinning is safe.

Isolation

We need isolation when we have shared resources and non-cooperative entities

As we increase isolation, we increase safety but lose usability and performance

We want to isolate kernel space from user space, user processes from each other, memory, security functions vs non-security functions

Isolation Mechanisms:

Protection Rings (**Isolate hardware**)

- Ring 0 is the highest privilege level that includes kernel. Protects shared resources in hardware like control registers, I/O port, memory access, important registers like %CR3 for paging and %CS for privilege level
- **System Calls** act as controlled entry points into kernel space**:**
  - Processes shouldn't be able to **directly** access kernel
  - Processes send system calls, which enter the kernel and switches privileges to execute the kernel service such as I/O, accessing hardware, killing processes, etc.
  - Kernel can validate arguments of sys-call and decide to allow/deny
- Address spaces (**Isolate memory**)
  - Every process should have identical virtual address space so they can't access other processes data
  - Each page table/directory entry has flags that display if it has user/kernel privileges and read/write privileges
- Time Slicing (Isolate CPU)
  - Abstract CPU so each process perceives its own CPU
  - Can be difficult to keep processes from inadvertently interacting

Intro to x86

Instruction Pointer
8 general purpose registers
6 segment registers
Status Registers
Control Registers
2^32 (4GB) Address Space
I/O Ports has space for 1024 interrupts
Protected Mode Segmentation
General Bootup Process?

Kernel Organization

Don't trust user processes, they'll fuck you up. They're filled with bugs and they're malicious lil bastards that will hog cpu and memory

Why are system calls made through software interrupts or traps instead of simple function calls?

- Isolation between kernel and user

Why do most OSes not allow user programs to directly access I/O ports or memory-mapped registers?

- User processes compete for hardware resources, so kernel should decide how to allocate resources fairly between processes

Why does each process have a separate virtual address space, instead of a shared one with per-page permissions?

- Simplicity: storing which processes have permissions for each page would be difficult to implement. Even if isolation can still be achieved with per-page permissions. It would be more difficult to implement how processes share and divide their virtual address spaces.

Why do OSes still rely heavily on hardware interrupts instead of event-driven software systems?

- Fully event-driven software would require polling, which would waste CPU.

	Monolithic Kernel	Microkernel
What it is	Entire OS resides in the kernel, so that the implementations of all system calls run in kernel mode.	Minimize the amount of operating system code that runs in kernel mode, and execute the bulk of the OS in user mode.
What runs in Kernel mode	Scheduling, memory management, file systems, networking	Minimal stuff like IPC for processes to communicate to each other, basic memory control, basic scheduling
Convenience	- Entire OS has full hardware privilege. - easier for different parts of OS to cooperate	For OS developer, easier to modularize and debug user-level services than kernel code
Performance	Faster because kernel processes can directly interact w/ each other	Slower from context switches and IPC overhead
Reliability	Error in kernel mode will cause all applications to fail	If it crashes, rest of kernel will be fine
Scalability	If more things are in kernel space, harder to scale because you gotta careful about shi	More modular so easier to evolve individual components because they are in user space

Virtual Memory

Virtual Memory Provides:

- Isolation, resource multiplexing (one physical address to many virtual address), allowing more virtual memory than we have RAM, protection

Processes should have the illusion of having access to the same memory regions

- They don't see physical memory; they see virtual memory
- MMU has a page table that maps VA → PA
- Page tables tell MMU what the mapping is
- Each process has their own page table, therefore their own VA → PA mapping

Why are page tables in kernel space:

- prevent user processes from tampering/accessing it
- Kernel is responsible for validating page table entries and swapping mappings
- MMU uses privileged registers

Virtual address → 20-bit page number and 12-bit offset into that page
Page Table Entry → Physical Page Number (PPN) and flags

- MMU translates virtual address using PPN to index into page table to find PTE

Most operating systems make far more sophisticated use of paging than xv6:

- demand paging from disk, copy-on-write fork, shared memory, lazily allocated pages, and automatically extending stacks

VM System should be able to support allocating new page tables and switching tables , also probably handling pagefaults and making sure that the read/write/execute are enforced?

Translation Lookaside Buffer:

- Stores address translations in case they are used again
- We have a separate TLB for data and instructions because they have different access patterns

Page Eviction Algorithm (from lecture):

- Have a circular list where you have a dirty hand and eviction hand.
- The dirty hand always points at the thing directly in front of the eviction hand and writes to disk if its dirty
- If we need to evict, the eviction pointer points to the page that was just written to disk by the dirty hand to evict it (since its safe to do so), and we move the dirty pointer forward.

Why every page table includes kernel mappings:

- Every user process has protected mapping of kernel page table in its own page table to avoid switching page tables on sys calls and interrupts
- Restricts process' direct access to the lower half of virtual memory
- Kernel needs to run instantly at any moment (sys calls, interrupts)
- If user process page tables only included user pages, we would need to switch to a kernel page table if an interrupt/sys-call came.
- This setup is convenient because after a process switches from user space to kernel space, the kernel can easily access user memory by reading memory locations directly.

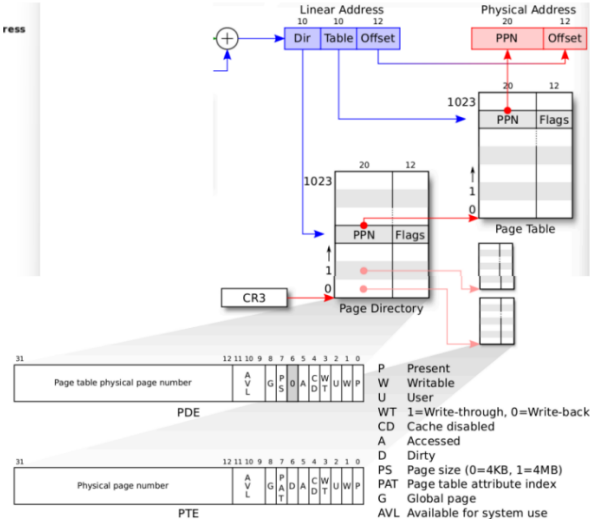
Paging vs Segmentation

- Segmentation is where physical memory is divided into different "segments" each with their own size.
- Segment registers would hold base address (base + offset) and linear address would hold offset
- Segmentation metadata is stored in a Global Descriptor Table
  - Holds segment information such as region bounds, protection (who can r/w) -> this allows our segment registers to store indexes into the GDT in protected mode (in 16bit mode we only get offsets)

Category	Segmentation	Paging
Memory Management	External fragmentation: segmentation requires memory to be allocated contiguously	- Does not require contiguous blocks to map memory - Limits external fragmentation - Efficiently Swaps pages to disk
Performance	Faster address translation	TLB misses cause extra memory accesses and page walks
Scalability	Not very scalable; requires register for each segment and caps out to 6 segments per process in x86	Very scalable, you can allocate memory for more processes than you have segment registers
Isolation (Per-process Protection)	Different protection per segment, segments that don't overlap have inherent isolation. Segments also can be shared if we like (multiplexing)	More granularity: more control of protection per-page since they're smaller

Purpose of Multi-level Page Tables:
- Reduce physical memory overhead for page tables
- Allow sparse address spaces without wasting memory
- Avoid requirement for contiguous physical memory for page tables
- Enable sharing of page tables between processes

Category	Single-Level Paging	Multi-Level Paging
Speed	Page translation only requires one lookup	Multiple page walks (memory reads) would need to occur for TLB miss
Memory Overhead	Very high — full page table must exist for *entire virtual address space*, even if most of it is unused. **Internal fragmentation**	Low / On-demand — only allocates page table entries for the portions of address space actually used. Allows smaller 4KB pages so less fragmentation
Fragmentation	No internal fragmentation inside page tables themselves (flat array), but page size still causes typical internal fragmentation in physical memory.	Slight additional fragmentation in page tables because each level is allocated in page-sized chunks even if not fully used — but overall *less wasted space* than single-level.
Scalability	Terrible — page table size grows exponentially with virtual address size. Completely impractical for 64-bit systems.	Excellent — scales to large virtual spaces efficiently by only populating tables as needed. This is why it's used in all modern 32/64-bit CPUs.
Isolation (Per-process Protection)	Supports isolation, since each process has its own full page table. Inefficient for large systems	More efficient sharing and stronger process memory isolation due to hierarchical address space



Why are page tables hierarchical (e.g., 2-level, 4-level) instead of flat?
- A flat page table for a 32-bit system (232) bytes of virtual address space with a 4KB page size would require over a million page table entries.
- Flat page tables require one entry for every virtual page.
- If each entry is 32-bit, the table alone would be 4MB (memory inefficient if a process uses only a tiny fraction of its address space
- With a 2-level table, the first-level table only needs to point to other second-level tables. If a user process doesn't use a large portion of its virtual address space, the corresponding second-level page table is never created, saving memory.

Why use Translation Lookaside Buffers (TLBs) instead of just caching page table entries in RAM?
- Latency: every memory access needs translation. Walking page tables in RAM would add several memory accesses; TLB lookup is on the critical path and is much faster (single cycle or a few cycles).

Why doesn't the OS store page tables in user space for faster lookups?
- user code could tamper with its own page tables to access other processes' memory or kernel memory. Page tables must be trusted data structures under kernel control.
- the kernel must manage and validate mappings, permissions, swap, and I/O interactions. If user code could change mappings arbitrarily, the kernel couldn't reliably enforce protection or maintain caches (TLBs).

Why is copy-on-write used for fork() instead of duplicating memory immediately?
- Most fork() + exec() programs call exec() immediately after fork(). If we copied memory eagerly, we'd waste time copying the parent's entire address space only to overwrite it.
- COW has isolation because on write the kernel allocates a private copy.

Why does the OS zero out freed memory pages before giving them to another process?
- Security: prevent leakage of previous process data (passwords, keys, plaintext) to the next process that receives the page.

Why are kernel stacks separate from user stacks?
- kernel stack runs in kernel privilege; separating it from the user stack prevents user code from overflowing into kernel frames and corrupting control flow or return addresses

Why are page faults handled in kernel mode even though they originate from user processes?
- The kernel must consult/modify page tables, allocate physical frames, maybe read from disk (swap/file), and update state atomically (all privileged operations).
- kernel must ensure security checks are enforced (permissions, copy-on-write) — user code must not be allowed to decide whether a fault should succeed

Design a hybrid paging and segmentation scheme that supports variable-sized segments but still uses fixed-size pages inside each segment.
- Design: Segments define logical units, paging handles physical memory
- Each segment has its own page table
- Create segment table mapping segment ID to descriptor w/ base address of page, segment limit in pages, and protection bits
- Virtual Address = (Segment Selector, Segment Offset) Segment Offset = (Page Number, Page Offset)

Suppose your system has limited physical memory — how would you decide which pages to swap out to disk?
- Evict clean pages first that are unused

Design a paging system that supports both user-level virtual memory isolation and shared memory between processes.
- Each process has private page tables that can share page table entires that map to the same physical frames (essentially just like fork)

Interrupts and Concurrency

Why must interrupt handling run in kernel mode?
- Requires unrestricted access to system hardware. Privileged mode is important for OS to safely handle hardware events without compromising security by allowing user processes to interfere

Why are interrupts typically disabled during parts of the scheduler or critical sections?
- Prevents race conditions. If interrupt were to occur and the interrupt handler edits the critical section, well then all shitters we got a race condition.

Why do OSes divide interrupt handling into top-half and bottom-half processing?
- When interrupt arrives, CPU stops what it was doing and jumps to interrupt handler
  - While in handler, nothing else can run so staying too long will hurt system responsiveness
- To solve issue, work is split into two parts:
  - top half runs immediately in the process's kernel thread on interrupt to handle urgent tasks.
  - Bottom half performs any work that can be deferred/schedulable after interrupts are re-enabled and wakes up the waiting processes.
- Allows device to service quickly without blocking other interrupts for too long

What are the advantages and disadvantages of using a single interrupt stack versus a per-interrupt stack?
- Single interrupt stack
  - Pros: memory efficient
  - Cons: no isolation between interrupt handlers
- Per interrupt stack
  - Pros: Better isolation and prevents stack overflow between handlers
  - Cons: Context swtch overhead between stacks and higher mem usage

Why can using spinlocks in int handlers be dangerous or lead to deadlock?
- Handlers can't sleep so they can't wait in a blocking queue
- If they spin on a lock that's held by code they interrupted, they'll spin forever
- Interrupts need to disabled before taking a spinlock (or just don't spinlock in interrupt)

Why is it unsafe for an interrupt handler to sleep or block?
- If a handler were to block, it would freeze the entire system because the process it interrupted would also be suspended, and the system would be unable to schedule the process or serve any lower-priority interrupts

How do you design condition variables that are safe to use both in interrupt and thread context?
- Interrupt handlers cannot block (unfinished answer)

How could interrupt storms (too frequent interrupts) starve user processes, and what kernel strategies reduce this?
- If frequent interrupts are starving user processes by hogging CPU time, we could either isolate user processes on separate cores than interrupts or batch multiple events into a single interrupt if possible

How can a poorly designed interrupt handler introduce priority inversion?
- If an interrupt handler tries to acquire a lock that a low-priority thread holds, then we're cooked. Just boost lock-holder's priority to solve this

Schedulers

	Pros	Cons
First In First Out	Simple to implement Predictable order Best for background workloads or batch workloads that care about throughput	Convoy effect: relatively short potential consumers of a resource get queued behind a long-running resource consumer
Shortest Job First	Minimizes average waiting time Predictable workloads for CPU bound	Starves long jobs, requires knowledge of job length
Round Robin	Best for fairness / responsiveness Good for time-sharing systems	Poor CPU utilization if small time slices; overhead from context switching
Priority Scheduling	Best for handling critical tasks	Starvation of low-priority tasks; prone to priority inversion
Multilevel Queue Scheduling	Best for systems with separated job classes (foreground vs background)	Hard to balance load between queues
Preemptive (Generally)	Ideal where responsiveness is critical and no process should monopolize CPU	More overhead from context switching

Waiting

When Sleeping:
- Address space: saved in process's page table
- User context: saved on kernel entry (system call)
- Kernel context: must be saved explicitly
- Process added to a wait list; can be woken up when condition met

	Sleep Lock	Spin Lock
If the Lock is Busy…	Process sleeps	Process spins (busy-waits)
CPU Usage	No CPU wasted	Wastes CPU cycles
Ideal For…	Long waits	Very short waits
Interrupt use?	no.	yes.
Overhead	Higher (context switch)	Shorter (if short wait)

Threading

Threads are a way to inform the OS that specific parts of a program can be executed concurrently: a basic unit of CPU utilization.
- Share the same view of memory as other threads in their thread group.

Why use threading?:
- Responsiveness
  - Especially for interactive applications
- Resource Sharing
  - Isolation is not necessary
  - Default definition of process defines separate address space; a thread shares memory and resources by default
- Economy
  - Creation / destruction is much faster than context switching, less overhead, etc.
- Multiprocessors
  - Perhaps achieve true parallelism vs. single-processor illusion

User Thread Tradeoffs:
- Pros: Efficient and flexible
  - Does not need user-kernel switching for data and can schedule threads freely
- Cons: Doesn't manage CPU resources and doesn't have direct control over sleeping
  - Needs kernel support for preemptive threading and using more than one CPU
  - Blocking can freeze all threads

Kernel Threading Models:
- Many **to One:** all user processes handled by a **single kernel thread**
  - Impractical on multicore or I/O bound systems
  - Two separate processes cannot share the kernel stack
- **One to one:** Each process has its own kernel stack and thread
- **Many** to **many:** Maps a pool of user threads to a pool of kernel threads
  - If kernel runs out of contexts, user programs must wait until one is free

Hybrid: allows the kernel to decide between many to many and one to one

	Pros	Cons
Many to One	Simple and efficient	If one thread blocks (e.g., on I/O), the entire process blocks
One to one	Parallelism on multiprocessor, Improved concurrency (even if we had one core, others threads can still run if one blocks for I/O)	Creating/destroying user threads is expensive since it requires system calls → overhead
Many to many	Balance of control and parallelism Fewer context switches	Kernel thread stalls can cause user-space thread blocking
Hybrid	Flexibility: you can bind critical threads, others multiplexed	Very complex, again, coordination between schedulers (scheduler activations)

Conditions for a deadlock
- Mutual Exclusion: At least one resource must be held in a non-sharable mode (only one thread can use it at a time).
- Hold and Wait: A thread must be holding at least one resource and waiting to acquire additional resources currently held by other threads.
- No Preemption: Resources cannot be forcibly taken away from a thread; they can only be released voluntarily.
- Circular Wait: There must exist a set of waiting threads $\{T_1, T_2, ..., T_n\}$ such that T_1 is waiting for a resource held by T_2, T_2 is waiting for a resource held by T_3, ..., and T_n is waiting for a resource held by T_1.

How can deadlock prevention, avoidance, and detection differ?
- Prevention:
  - Design the system to ensure that at least one of the four necessary conditions for deadlock can never hold.
  - Negate Hold & Wait (acquire all resources at once, or none).
  - Allow Preemption.
  - Negate Circular Wait (impose a strict ordering on resource acquisition).

- Avoidance:
  - The OS is given advance knowledge of the maximum resources each process might request. It dynamically checks if a resource allocation could lead to a future deadlock.
  - The OS uses an algorithm (like the Banker's Algorithm) to decide if granting a request leads to a "safe state" (a state where deadlock is impossible). If not, it delays the request.

- Detection and Recovery:
  - The OS doesn't try to prevent deadlocks. Instead, it periodically checks (using a resource allocation graph) to see if a deadlock has occurred.
  - 1. Detection Algorithm: Examines the system state for a circular wait.
  - 2. Recovery: When detected, it breaks the deadlock by terminating processes or preempting (rolling back) resources.

	Pros	Cons
Prevention	Guarantees no deadlocks.	Reduced resource utilization and system throughput.
Avoidance	More flexible than prevention; allows for higher resource utilization.	Requires knowledge of future resource needs, which is often impractical.
Detection and Recovery	Minimal runtime overhead during normal operation.	The cost of recovery can be high (lost work from termination/rollback).

Explain the producer–consumer problem and how semaphores solve it.
- Popular synchronization problems:
  - Producer(s): generate data items and place them in a shared buffer
  - Consumer(s): remove items from the buffer and process them
- The buffer has bounded capacity (size N).
  - Producers must wait if the buffer is full (can't over-produce)
  - Consumers must wait if the buffer is empty (nothing to consume)
- Access to the buffer (insertion, removal, index increment, etc.) must be mutually exclusive to avoid race conditions

```
int consume() {
    lock(lk);
    while (q.empty()) {
        cv.wait(lk);
    }
    v = q.pop();
    unlock(lk);
    return v;
}
```

Implement a dining philosophers solution that avoids deadlock.
- Number the forks 1-5. Philosophers must always pick up the lower-numbered fork first, except the last philosopher who picks up forks in reverse order. This breaks the circular wait by establishing a global ordering.

```
produce(int v) {
    lock(lk);
    q.push(v);
    unlock(lk);
    cv.signal();  // Signal after mutex is released
}
```

Suppose two threads increment a shared integer variable 1000 times each — what's the possible final range of values if no synchronization is used?
- 1000-2000. The minimum occurs when both threads read the same value simultaneously and both write back value+1, losing one increment. Perfect interleaving gives 2000.

How would you implement a thread-safe queue?
- Check Consumer-Producer problem

Show how condition variables can coordinate producer–consumer interactions.
- Producers acquire mutex, check if buffer full → wait on "not_full" condition, add item, signal "not_empty". Consumers acquire mutex, check if buffer empty → wait on "not_empty", remove item, signal "not_full". Always use while loops to handle spurious wakeups.

How would you detect a deadlock among waiting threads programmatically?
- Maintain a wait-for graph where nodes are threads and edges represent "thread A waiting for resource held by thread B". Periodically run cycle

detection using DFS. When a cycle is detected, choose the victim thread based on priority/age and terminate it or force resource release.

How would you design a thread pool to manage incoming tasks efficiently?
- Fixed number of worker threads pull tasks from a **shared blocking queue**. Main thread submits tasks to queue. Workers continuously dequeue and execute tasks.
- Address same concerns in Consumer-Producer problem

Design a scheduler for threads that ensures fairness and avoids starvation.
- Multi-level feedback queue with aging. Multiple priority levels with round-robin within each level. Threads demoted after using time quantum, promoted if waiting too long. Prevent starvation of low-priority threads like aging as well.

Suppose you're implementing user-level threads — how do you handle blocking system calls?
- For I/O operations, use non-blocking system calls and readiness notification (epoll/kqueue). When a thread would block, save its state and switch to another runnable thread. Resume when I/O completes.

How would you implement priority scheduling for threads?
- Multiple ready queues per priority level. Scheduler always picks highest-priority runnable thread. To prevent priority inversion, when high-priority thread waits for lock held by low-priority thread, temporarily boost holder's priority.

Design a multithreaded web server — how would it handle multiple clients concurrently?
- Acceptor thread accepts connections and hands them to worker threads via queue. Each worker thread handles entire request. Limit concurrent connections per worker, and handle producer-comsumer problem.

Compare a thread-per-request model vs. an event-driven model for servers.
- Thread-per-request: Simple synchronous programming, good for CPU-bound tasks, but poor scalability due to thread overhead and memory usage.
- Event-driven: Single thread with event loop handles all I/O. Excellent for I/O-bound tasks, high scalability, but poor for CPU-bound work.
- Hybrid Approach: Small thread pool for I/O with event loop, separate pool for CPU work. Best of both worlds.

How would you design a work-stealing scheduler for a multi-core environment?
- Each processor has its own **deque**. Workers push/pop from their own queue's front. When a worker runs out of work, it steals from the back of another random worker's queue. This provides good load balancing with low contention.

How does the OS handle kernel thread scheduling on multiple cores?
- Per-CPU run queues with work stealing. Load balancer periodically redistributes threads. Prefer scheduling threads on cores where they last ran.